arXiv:2412.18955v1 [cs.SD] 25 Dec 2024

Leave-One-EquiVariant: Alleviating invariance-related information loss in contrastive music representations

Julien Guinot*[1,2], Elio Quinton[2], and György Fazekas[1]

[1]*School of EECS, Queen Mary University of London, London, U.K*
[2]*Music & Audio Machine Learning Lab, Universal Music Group, London, U.K*

***Abstract*—Contrastive learning has proven effective in self-supervised musical representation learning, particularly for Music Information Retrieval (MIR) tasks. However, reliance on augmentation chains for contrastive view generation and the resulting learnt invariances pose challenges when different downstream tasks require sensitivity to certain musical attributes. To address this, we propose the Leave One EquiVariant (LOEV) framework, which introduces a flexible, task-adaptive approach compared to previous work by selectively preserving information about specific augmentations, allowing the model to maintain task-relevant equivariances. We demonstrate that LOEV alleviates information loss related to learned invariances, improving performance on augmentation related tasks and retrieval without sacrificing general representation quality. Furthermore, we introduce a variant of LOEV, LOEV++, which builds a disentangled latent space by design in a self-supervised manner, and enables targeted retrieval based on augmentation related attributes.**

***Index Terms*—Contrastive Learning, Music Representations**

I. Introduction, Related work

Contrastive learning is a powerful paradigm for learning self-supervised representations. These representations have been proven to be effective on a range of downstream tasks, including in MIR. Since its first adaptation from computer vision [1] by *Spijkervet et al.* [2], contrastive learning of musical representations has been successfully repurposed with different architectures [3], [4], notably MULE [5]), and positive mining strategies [6]–[10]. Despite its success, the performance of contrastively learned representations on downstream tasks is sensitive to the positive mining strategy and the augmentation chain. Training a contrastive model involves maximizing (resp. minimizing) agreement between positive (resp. negative) augmented samples. Thus, the choice of positive sampling strategy and the augmentations the model learns to "ignore" significantly impact the usefulness of learnt representations w.r.t. downstream tasks. For example, in music, genre is transposition invariant: transposing a piece of music does not change its genre. So, including pitch shifting in contrastive training should benefit genre recognition by teaching the model to recognise similar representations across different keys. However, this would harm performance on tasks such as pitch detection, chord recognition or key detection, which rely on key-dependent features. Incorporating an augmentation in training renders representations invariant to this transformation, which may or may not be useful depending on the task. Furthermore, the positive sampling strategy implicitly guides the notion of similarity. That is, a contrastive model trained in this framework is never truly generic, and learnt invariances and similarities might not be task-appropriate for downstream applications.

In practice and to the best of our knowledge, there exists no unified understanding or theory of how the augmentation chain and the positive mining strategy influence downstream performance or each other, nor any one-size-fits-all recipe for contrastive learning approaches. Some works have attempted to alleviate these issues by devising better positive mining strategies [11], reducing false negatives within the mining strategy [12], [13], or influencing positivity and negativity with semantic weighing [8], [11], [14]. From the standpoint of the augmentation chain, a body of work on understanding the effect of certain augmentations on downstream tasks exists [15], [16], and similar studies have started to appear in MIR [17]. Information-theoretical work has attempted to understand the interplay of positive mining and augmentation chains in training contrastive models [15]. One work of particular interest to this study, Leave One Out Contrastive (LOOC) [16], proposes an approach that does not discard information related to pre-training augmentations in representations. Our work adapts this paradigm to MIR by alleviating learnt invariances pertaining to musical pitch and tempo. Briefly, our contributions are as follows.

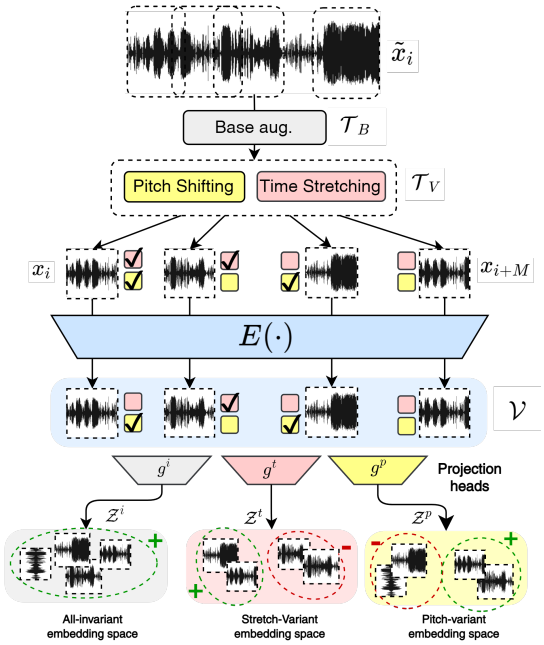

Fig. 1: Leave One EquiVariant framework. Subspace $\mathcal{Z}^k$ is invariant to all transformations except T_k, forcing the embedding superspace $\mathcal{V}$ to conserve information about all transformations.

- We introduce Leave One EquiVariant (LOEV) and its variant LOEV++, an adaptation of the Leave One Out Contrastive (LOOC) framework, to mitigate invariance-related information loss with greater flexibility.
- LOEV(++) enhances performance on augmentation-related tasks and retrieval without compromising general representation quality.
- LOEV++ creates a disentangled latent space, enabling targeted retrieval and controlled storage of augmentation information.
- Unlike [16], where links between augmentations and downstream task are unclear, we demonstrate that LOEV(++) reduces perfor-

mance loss for explicitly linked augmentations and MIR tasks.

II. Methods

A. Contrastive embedding space

In the traditional contrastive learning framework, consider a batch of non-augmented samples $\{\tilde{x}_i\}, i \in [1...N]$. These samples are augmented through an ordered stochastic chain of K nuclear augmentations $\{T_k\}, k \in [1...K]$. These nuclear augmentations have a probability p_k of being applied. Parameters for each T_k are randomised for each augmented sample. Non-augmented samples $\tilde{\mathbf{x}}$ are augmented M times into samples $\mathbf{x}_i = \{x_i\}, i \in [1...NM]$. The set of positives $P(i)$ for a given index i is the set of samples originating from the same original sample x_i. The contrastive objective is given by the following loss for a sample i:

$$\mathcal{L}_i = \frac{-1}{|P(i)|} \sum_{p \in P(i)} \frac{\exp(sim(z_i, z_p)/\tau)}{\sum_{j \neq i} \exp(sim(z_i, z_j)/\tau)} \tag{1}$$

z_i is the output of an encoder $E : x \mapsto e \in \mathbb{R}^{d_E}$ and a projection head $g : e \mapsto z \in \mathcal{Z}^i \subset \mathbb{R}^{d_g}$, τ is a temperature hyperparameter.

In this framework, the network is taught to be invariant to all transformations (all-invariant) by mapping augmented samples to one embedding. We propose a novel, ad-hoc augmentation tracking framework that can be added to existing contrastive learning pipelines where the model maintains information about all augmentations of interest - **L**eave **O**ne **E**qui**V**ariant (LOEV). In addition to an all-invariant projection head g^i into an embedding space $\mathcal{Z}^i$, K projection heads g_k project the hidden representation $e \in \mathcal{V}$ onto K spaces $\{\mathcal{Z}^k\}$. representations in $\mathcal{Z}^k$ are invariant to all transformations except T_k. For each subspace to maintain variance w.r.t a transformation, the set of positives $P_k(i)$ for sample i w.r.t T_k is samples from the same anchor that have *not* been augmented with T_k. This way, each projection space is explicitly taught to be variant to *one* augmentation, and the embedding space $\mathcal{V}$ must contain information about all transformations to minimize all objectives. We employ a set of *base* augmentations $\mathcal{T}_B$ without a dedicated subspace (i.e. the model learns to be invariant to them in all subspaces) and *variant* augmentations $\mathcal{T}_V$ which each have a dedicated variant subspace. The contrastive objective for variant subspace $\mathcal{Z}^k$ is:

$$\mathcal{L}_i^k = \frac{-1}{|P_k(i)|} \sum_{p \in P_k(i)} \frac{\exp(sim(z_i, z_p)/\tau)}{\sum_{j \neq i} \exp(sim(z_i, z_j)/\tau)} \tag{2}$$

The global loss is then averaged over all subspaces:

$$\mathcal{L}_i^T = \frac{1}{K+1} \left(\mathcal{L}_i + \sum_{k=0}^{K} \mathcal{L}_i^k \right) \tag{3}$$

B. View generation

To generate positive views, Leave-one-out contrastive (LOOC) [16], keeps the clean anchor as a query; After generating an augmented sample, parameters from each nuclear augmentation except one per view are copied across K additional views. Our framework is similar in spirit but more flexible, employing an ad-hoc positive tracking method as well as a simCLR [1] contrastive pipeline versus MoCo [18] in [16]. Instead of always applying all augmentations but one for K augmentations, which induces some inflexibility to the implementation, we maintain our stochastic augmentation chain and simply track which augmentations are applied to each sample i within a binary vector $t_i \in \{0, 1\}^K$ ($t_{i,k} = 1$ if augmentation k is applied to sample i else 0). For a given augmentation T_k, the set of positives is $P_k(i) = P(i) \cap \{j | t_{j,k} = t_{i,k} = 0\}$. If two samples have been augmented with the target augmentation, we consider that stochastic uniform sampling of continuous parameters of the augmentation is a sufficient guarantee that the embedding should *not* be the same for both views.

C. Model input, pretraining augmentation chain

Our model (Section II-D) takes as inputs log-scaled mel-spectrograms. We sample 3 seconds of 16kHz mono audio and convert it to a Log-Mel Spectrogram. Prior to the augmentation chain, M chunks of 3 seconds are sampled from the track according to 3 strategies and then augmented: "*Same*" strategy samples the same chunk M times. "*Adjacent*' strategy samples M adjacent chunks with no overlap. "*Random*" strategy randomly samples M chunks from the track, allowing for overlap. These strategies accommodate previous findings that different positive positions in the track lead to different representation strengths [7]. As key and tempo are *relatively* position-invariant within a track, we assume this will not have an impact on learned representations. $\mathcal{T}_B$ includes Gain, Polarity Inversion, Colored Noise addition, Filtering (one of low / high passing, or band cut / passing), Reverb, and Distortion. $\mathcal{T}_V$ includes Pitch shifting continuously between -4 and 4 semitones, and Continuous time stretching with factors sampled between 0.7 and 1.3 - Both applied with 50% chance. We augment each anchor sample 4 times.

D. Model architecture

As in [5], [19], we leverage a F0-SF-NFNet as the encoder for our contrastive task. Official implementations are available only in Jax [*] and Keras [†], so we reproduce the model in Pytorch. In MULE [5], the projection head is a 47M parameter 3-layer MLP. Because we use multiple projection heads, we scale back our projection head sizes to two 2048-wide hidden layers. MULE benefits from large-scale high-quality pretraining data, which allows it to forgo traditional augmentation approaches and limit itself to an efficient in-batch mixup approach. We *do* use other augmentations (Section II-C).

As in [16], we implement variations of MULE and LOEV, namely MULE++ and LOEV++. Authors in [16] introduce LOOC++, which integrates the last convolutional block of the ResNet50 backbone into the projection heads, creating multiple $\mathcal{V}$ superspaces and disentangling in the global latent space into smaller transformation-variant spaces. We do the same for LOEV++ by resorbing a 10M parameter portion of the network into the projection heads. For the all-invariant head, the pitch variant head and the stretch-variant head (resp. g^i, g^p, g^t) the spaces prior to the resorbed portion of the model are notated $\mathcal{V}^i, \mathcal{V}^t, \mathcal{V}^p$, and can all be used as frozen representation spaces (See Sec. III-D). The concatenation of these spaces is $\mathcal{V}^{++}$

III. Experiments and results

We study two variant augmentations, pitch shifting (PS) and time stretching (TS). Both these augmentations are single-parameter transformations which are directly correlated to semantic musical information (pitch and key, tempo). These choices illustrate the effectiveness of our method for explicitly semantic audio transformations, but the method itself is transformation-agnostic.

A. Datasets and evaluation metrics

We use the *MTG-Jamendo dataset* [20] for pretraining. We evaluate pretrained models on automatic tagging, our proxy task for general music understanding, by predicting the top 50 tags for Jamendo and *MagnaTagATune* [21] with canonical data splits [2], [22]. We

[*] https://github.com/google-deepmind/slowfast_nfnets
[†] https://github.com/PandoraMedia/music-audio-representations

	Aug				MTT50		Jam50		Giansteps	NSynth	AllTempo	
	Mixup	B	PS	TS	AUROC	AP	AUROC	AP	Acc_w	Acc	Acc1	Acc2
MULE	✓				89.4 (↓)	36.4 (↓)	82.0	26.6	51.9 (↓)	84.0 (↓)	69.3 (↓)	90.2 (↓)
		✓			89.7 (B)	37.3 (B)	82.3	27.3	57.4 (B)	85.4 (B)	70.4 (B)	92.3 (B)
		✓	✓		90.1 (↑)	38.0 (↑)	82.3	27.3	16.7 (↓)	74.8 (↓)	**72.1** (↑)	**92.6** (↑)
		✓		✓	90.3 (↑)	38.4 (↑)	82.4	27.3	**64.7** (↑)	**89.2** (↑)	36.0 (↓)	44.7 (↓)
		✓	✓	✓	**90.6** (↑)	**38.7** (↑)	**82.6**	**27.7**	15.9 (↓)	80.1 (↓)	65.1 (↓)	85.3 (↓)
MULE++		✓	✓	✓	90.5 (↑)	38.4 (↑)	82.7	27.8	15.1 (↓)	78.8 (↓)	63.2 (↓)	84.1 (↓)
LOEV		✓	✓		90.0 (↑)	38.0 (↑)	82.4	27.3	38.9 (↓)	84.0 (↓)	**71.3** (↑)	**91.9** (↓)
		✓		✓	90.0 (↑)	37.8 (↑)	82.3	27.2	**60.0** (↑)	**88.1** (↑)	71.2 (↑)	90.8 (↓)
		✓	✓	✓	**90.5** (↑)	**38.4** (↑)	**82.6**	**27.7**	40.0 (↓)	83.5 (↓)	70.0 (↓)	90.3 (↓)
LOEV++		✓	✓	✓	90.6 (↑)	38.4 (↑)	82.7	27.8	44.2 (↓)	84.5 (↓)	72.6 (↑)	91.1 (↓)
MULE-L [5]	✓				91.4	40.4			66.7	89.2	-	-
MULE-S [5]	✓				90.5	38.9			50.8	82.4	-	-

TABLE I: Results for downstream probing on automatic tagging (MagnaTagATune, Jam50), Key (Giantsteps) and pitch (NSynth) estimation and tempo estimation (AllTempo). Baseline is denoted by (B). Performance improvements (resp. losses) are denoted by (↑) (resp. (↓)).

report AUROC and mean Average Precision. Two straightforward transposition-variant tasks are key estimation and pitch estimation. Key estimation is formulated as a 24-way classification task with the *Giantsteps* dataset [23] for training and *MTG-Giantsteps* [24] for testing, as in [25]. The metric for key estimation is a weighted accuracy taking into account reasonable errors [26]. We use the *mir-eval* implementation. We employ *NSynth* [27] for pitch estimation, the metric is accuracy over 112 pitch classes. Tempo estimation is our stretch-variant task. Four datasets : *GTZAN* [28], *ACM-MIRUM* [29], *Hainsworth* [30] and *Giantsteps* [23] are used in a one-vs-all fashion, i.e., when testing on one dataset, we train on all 3 other datasets — we call this dataset *AllTempo* as in [31]. During probe training, We implement a time-stretching augmentation with a stretching rate $\tau \sim \mathcal{U}(0.8, 1.2)$ for robustness [31], [32]. The metrics for this task are 300-class acc_1 and acc_2, two tolerance-reinforced accuracy metrics [26]. acc_2 allows for octave errors, i.e. predictions within a tolerance interval around reasonable ratios of the ground truth.

B. Pretraining details

We pretrain our models on the MTG-Jamendo dataset for 200000 steps with the Adam optimiser. The authors of [5] use a cosine decay learning rate scheduler with linear warmup. We find that in our case, perhaps due to the reduced batch size, such scheduling leads to slightly worse results across the board, so we train with a constant learning rate of 0.0001. The authors in [5] propose two MULE models trained on proprietary datasets and batch sizes of different scales. We pretrain on comparatively smaller batch sizes (256 vs MULE-Large's 3840 and MULE-Small's 512) and data scale (2.9k hours vs 5k — MULE-S and 117.5k — MULE-L). This partly explains the discrepancies between our results and those reported in [5]. Table I shows that including time stretching and pitch shifting in the pretraining augmentations compensates for the performance loss due to this down-scaling and is thus an attractive strategy.

We pretrain the following model-augmentation chain combinations and use their frozen representations from $\mathcal{V}$ or $\mathcal{V}^{++}$ in Sections III-C,III-D and III-E: A MULE model trained with only Mixup as the augmentation pipeline, as in the original implementation [5]. The parameters for the mixup gain are taken from [5]. MULE and LOEV models trained with the augmentation chain specified in Section II-C. Used pretraining augmentation chains include *without* variant augmentations (B), *with either* variant augmentations (PS or TS), and *with both* variant augmentations (PSTS). When only one variant augmentation is present, the projection head for the missing variant augmentation is not considered in the loss computation. Finally, we pretrain LOEV++ and MULE++ models with PSTS.

C. Downstream probing evaluation

For our first set of experiments, we probe frozen representations on tasks pertaining to pitch shifting and time stretching: Tempo estimation, key and pitch estimation, and automatic tagging (See Sec. III-A). We train shallow Relu-nonlinear MLPs with different layer depths, widths, and dropout values on frozen representations for each task. We employ the Adam optimizer to train the probes. Dropout, learning rate, and probe architecture are empirically adjusted for each dataset to avoid overfitting. We train on chunk-level embeddings, and average embeddings across the full track for test-time inference, as is customary. Results are reported in Table I.

As in previous studies [16], [17], including augmentations in the augmentation chain for the standard contrastive learning method is harmful to tasks that require that information, e.g. pitch shifting with key estimation and time stretching with tempo estimation. (See Table I) In counterpart, including these augmentations is beneficial to tasks such as automatic tagging that are key and pitch-invariant to an extent. Note that MULE-Mixup results are always slightly worse than results for MULE-B because our base augmentation chain is *non-destructive* to key and tempo, whereas mixup *is* . The tradeoff between general and task-specific performance is alleviated by LOEV: even with harmful augmentations, the performance drop is drastically reduced. leaving the transformation out entirely still leads to better performance, arguably due to design considerations and hyperparameters for our positive selection strategy. One anomaly is the performance of MULE-TSPS on tempo estimation, which surprisingly does not suffer as much as with MULE-TS. Arguably, a frequency-wise shift is an easier identifiable transformation than a time-wise stretch on 3-second snippets, which the model might be overfitting to. This warrants further investigation.

D. Representation Embedding space for downstream probing

We evaluate the performance of LOEV++ (PSTS) when compared to MULE++ (PSTS) on the same downstream tasks as previously, when probing different subspaces. We report performance on MTAT, average performance over AllTempo and key estimation performance. Results are reported Table II. Furthermore, we report performance for LOEV++ when probing $\mathcal{V}^i$, $\mathcal{V}^p$, $\mathcal{V}^t$. MULE++ is trained with the same head duplication scheme as LOEV++ but the same objective for all heads, as in [16]. With this, the probing spaces for MULE++ and LOEV++ are equally sized for fair comparison.

We also aim to understand how the information relating to different transformations is stored within the embeddings. We iterate over the MTAT test set and apply both time stretching and pitch shifting. We average the non-augmented and the augmented chunk embeddings z and $z^\star$ over tracks, for different subspaces. We compute the

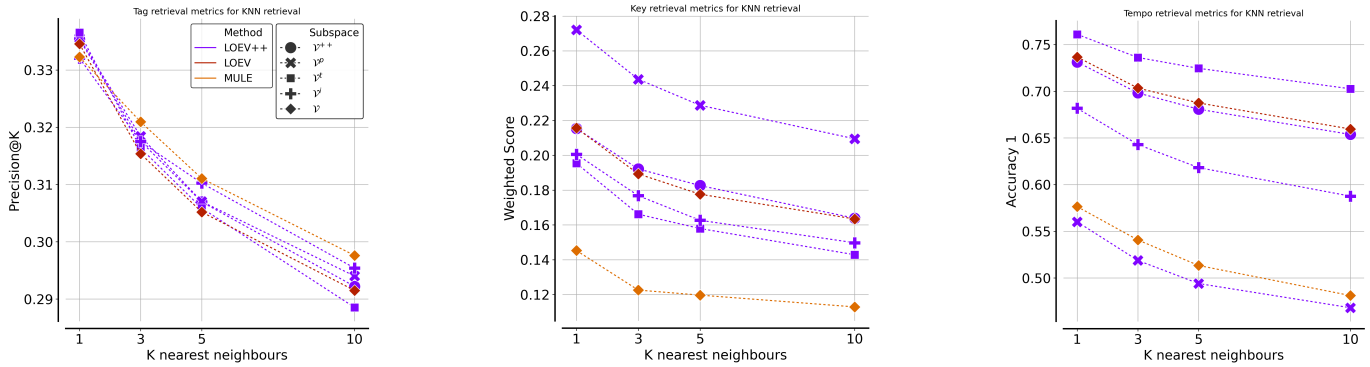

Fig. 2: Retrieval metrics for retrieved tags (MagnaTagATune), key (Giantsteps), and tempo (AllTempo) - Precision@K, Weighted accuracy and acc_1 are computed between the seed track embedding and the retrieved labels for the $k \in [1, 3, 5, 10]$ nearest neighbouring embeddings.

		MTAT		Giantsteps	AllTempo	
Model	Subspace	AUROC	AP	Acc_w	Acc1	Acc2
MULE++	$\mathcal{V}^{++}$	90.5	38.4	15.1	63.2	84.1
	$\mathcal{V}^{++}$	**90.6**	**38.4**	**44.2**	**72.6**	91.1
	$\mathcal{V}^{i}$	**90.5**	**38.4**	39.0	70.7	89.4
	$\mathcal{V}^{p}$	90.4	38.3	**43.0**	64.7	84.3
LOEV++	$\mathcal{V}^{t}$	90.3	38.2	30.0	**71.5**	**91.2**

TABLE II: Probing subspace experiment results. we compare probing results on automatic tagging, key estimation and tempo estimation while probing different subspaces of $\mathcal{V}^{++}$, namely $\mathcal{V}^{i}$,$\mathcal{V}^{p}$,$\mathcal{V}^{t}$

cosine distance d_c between clean and transformed embeddings: $d_c(z, z^\star) = 1 - (z \cdot z^\star)/(||z||||z^\star||)$ as in [17]. The evolution of this cosine distance with the number of semitones applied for pitch shifting is shown in Fig. 3 for MULE and LOEV with various pretraining augmentations and for different subspaces of LOEV++[‡].

Similar to results presented in [17], we find that both MULE-TS and LOEV-TS present more marked fluctuation of cosine distance when pitch shifting the audio than pretrained with PS or PSTS, due to learned invariances. We notice that $d_c(z, z^\star)$ is higher for MULE than for LOEV, while the general structure is conserved, explaining differences in performance in Table I. We verify musically plausible results shown in [17] showing noticeable dips in cosine distance at harmonic series pitch shifting factors . In the latent space of LOEV++, we find a similar pitch-variant structure in the $\mathcal{V}^p$ subspace despite training with PSTS, showing that pitch-variant information is well maintained in this space, and confirming findings in Table II. $d_c(z, z^\star)$ in other subspaces is smoother and resembles $d_c(z, z^\star)$ found in $\mathcal{V}$ for LOEV and MULE PS or PSTS.

E. Retrieval experiments

To verify that retrieval in the latent space preserves semantic information relating to the applied variant transformations, we run retrieval experiments on our key, tempo, and tagging datasets. We retrieve test set nearest neighbours using cosine distance (KNN with $k \in [1, 3, 5, 10]$) and compute relevant metrics (Table III-A) between query and retrieved data points averaged over queries. For tempo datasets, We perform KNN retrieval on all 4 datasets simultaneously. We also retrieve KNN tags from the MTAT test set and compute precision @k for each tag, averaged over tags. The intuition behind these experiments is that nearest neighbours in a space which discards information related to a given semantic aspect will not be neighbours based on that information. These experiments are of interest as one of

[‡]We choose to represent pitch shifting as it is more visually explicit, but perform the same experiments for time stretching and find similar results.

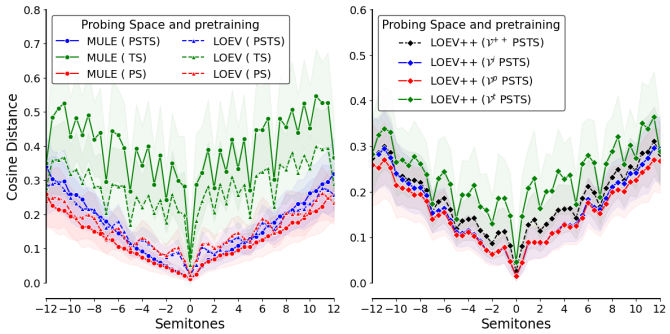

Fig. 3: Cosine distance between embeddings of pitch-shifted and non-pitched audio snippets for LOEV and MULE (left) and LOEV++ (right) and different subspaces and pretraining.

our focuses with this study is manipulating semantic retrieval in the latent space. Fig. 2 reports results for tempo, key, and tag retrieval.

We find that while MULE-PSTS maintains an edge on tag retrieval, it is largely outperformed by both LOEV and LOEV++ subspaces for tempo and key retrieval. Finally, We find that LOEV++ is also effective in disentangling information in different subspaces for retrieval. Key retrieval yields the best result in the pitch-variant space $\mathcal{V}^p$, and similarly $\mathcal{V}^t$ performs the best for tempo retrieval.

IV. Conclusion and Future Work

This paper introduced LOEV(++), a novel method for contrastive representation learning, applied to contrastive learning of musical representations. By incorporating adaptive ad-hoc augmentation tracking and specific augmentation-variant subspaces and training objectives during pretraining, LOEV(++) effectively reduces information loss related to learned invariances in both shallow probing and retrieval tasks with no detriment to tagging performance. LOEV(++) offers performance gains over MULE-PSTS while remaining computationally sober compared to approaching similar performance With MULE-Mixup. We show that LOEV++ explicitly disentangles augmentation information in the latent space, enabling attribute-targeted retrieval and competitive performance on downstream tasks.

This study primarily focused on audio transformations related to key, pitch, and tempo, chosen for their direct links to semantic information. While ideal for demonstrating our method's effectiveness, other transformations such as distortion, reverb, and compression, as well as nonparametric semantic transformations relating to genre or instrumentation, were not explored and are left to future work.

REFERENCES

[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, et al., "A simple framework for contrastive learning of visual representations," in *International conference on machine learning*. PMLR, 2020, pp. 1597–1607.

[2] Janne Spijkervet and John Ashley Burgoyne, "Contrastive Learning of Musical Representations," in *International Society for Music Information Retrieval (ISMIR)*. Sept. 2021, number arXiv:2103.09410, arXiv.

[3] Hang Zhao, Chen Zhang, Bilei Zhu, et al., "S3t: Self-supervised pre-training with swin transformer for music classification," in *ICASSP 2022-2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2022, pp. 606–610.

[4] Marcel A Vélez Vásquez and John Ashley Burgoyne, "Tailed u-net: Multi-scale music representation learning.," in *ISMIR*, 2022, pp. 67–75.

[5] Matthew C. McCallum, Filip Korzeniowski, Sergio Oramas, et al., "Supervised and Unsupervised Learning of Audio Representations for Music Understanding," in *International Society for Music Information Retrieval (ISMIR)*. Oct. 2022, number arXiv:2210.03799, arXiv.

[6] Christos Garoufis, Athanasia Zlatintsi, and Petros Maragos, "Multi-Source Contrastive Learning from Musical Audio," May 2023, number arXiv:2302.07077, arXiv.

[7] Jeong Choi, Seongwon Jang, Hyunsouk Cho, and Sehee Chung, "Towards proper contrastive self-supervised learning strategies for music audio representation," in *2022 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2022, pp. 1–6.

[8] Julien Guinot, Elio Quinton, and György Fazekas, "Semi-supervised contrastive learning of musical representations," *arXiv preprint arXiv:2407.13840*, 2024.

[9] Dong Yao, Zhou Zhao, Shengyu Zhang, et al., "Contrastive Learning with Positive-Negative Frame Mask for Music Representation," in *Proceedings of the ACM Web Conference 2022*, Apr. 2022, pp. 2906–2915.

[10] Ruben Ciranni, Emilian Postolache, Giorgio Mariani, Michele Mancusi, Luca Cosmo, and Emanuele Rodolà, "COCOLA: Coherence-Oriented Contrastive Learning of Musical Audio Representations," Apr. 2024.

[11] Fan Yang, Kai Wu, Shuyi Zhang, Guannan Jiang, Yong Liu, Feng Zheng, Wei Zhang, Chengjie Wang, and Long Zeng, "Class-Aware Contrastive Semi-Supervised Learning," Sept. 2022.

[12] Tri Huynh, Simon Kornblith, Matthew R. Walter, Michael Maire, and Maryam Khademi, "Boosting Contrastive Self-Supervised Learning with False Negative Cancellation," in *2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Waikoloa, HI, USA, Jan. 2022, pp. 986–996, IEEE.

[13] Songwei Ge, Shlok Mishra, Chun-Liang Li, Haohan Wang, and David Jacobs, "Robust Contrastive Learning Using Negative Samples with Diminished Semantics," in *Advances in Neural Information Processing Systems*. 2021, vol. 34, pp. 27356–27368, Curran Associates, Inc.

[14] Ho-Hsiang Wu, Prem Seetharaman, Kundan Kumar, et al., "Wav2CLIP: Learning Robust Audio Representations from Clip," in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, May 2022, pp. 4563–4567.

[15] Yonglong Tian, Chen Sun, Ben Poole, Dilip Krishnan, Cordelia Schmid, and Phillip Isola, "What Makes for Good Views for Contrastive Learning?," in *Advances in Neural Information Processing Systems*. 2020, vol. 33, pp. 6827–6839, Curran Associates, Inc.

[16] Tete Xiao, Xiaolong Wang, Alexei A. Efros, and Trevor Darrell, "What Should Not Be Contrastive in Contrastive Learning," Mar. 2021.

[17] Matthew C McCallum, Matthew EP Davies, Florian Henkel, Jaehun Kim, and Samuel E Sandberg, "On the effect of data-augmentation on local embedding properties in the contrastive learning of music audio representations," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 671–675.

[18] Xinlei Chen, Haoqi Fan, Ross Girshick, et al., "Improved Baselines with Momentum Contrastive Learning," Mar. 2020.

[19] Luyu Wang, Pauline Luc, Yan Wu, et al., "Towards Learning Universal Audio Representations," in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, May 2022, pp. 4593–4597.

[20] Dmitry Bogdanov, Minz Won, Philip Tovstogan, et al., "The mtg-jamendo dataset for automatic music tagging," in *Machine Learning for Music Discovery Workshop, International Conference on Machine Learning (ICML 2019)*, Long Beach, CA, United States, 2019.

[21] Edith Law, Kris West, Michael I Mandel, et al., "Evaluation of algorithms using games: The case of music tagging.," in *ISMIR*. Citeseer, 2009, pp. 387–392.

[22] Jordi Pons, Oriol Nieto, Matthew Prockup, Erik Schmidt, Andreas Ehmann, and Xavier Serra, "End-to-end learning for music audio tagging at scale," *arXiv preprint arXiv:1711.02520*, 2017.

[23] Peter Knees, Ángel Faraldo Pérez, Herrera Boyer, Richard Vogl, Sebastian Böck, Florian Hörschläger, Mickael Le Goff, et al., "Two data sets for tempo estimation and key detection in electronic dance music annotated from user corrections," in *Proceedings of the 16th International Society for Music Information Retrieval Conference (ISMIR); 2015 Oct 26-30; Málaga, Spain.[Málaga]: International Society for Music Information Retrieval, 2015. p. 364-70.* International Society for Music Information Retrieval (ISMIR), 2015.

[24] Filip Korzeniowski and Gerhard Widmer, "End-to-end musical key estimation using a convolutional neural network," in *2017 25th European Signal Processing Conference (EUSIPCO)*. IEEE, 2017, pp. 966–970.

[25] Ruibin Yuan, Yinghao Ma, Yizhi Li, et al., "MARBLE: Music Audio Representation Benchmark for Universal Evaluation," July 2023.

[26] Colin Raffel, Brian McFee, Eric J Humphrey, Justin Salamon, Oriol Nieto, Dawen Liang, Daniel PW Ellis, and C Colin Raffel, "Mir_eval: A transparent implementation of common mir metrics.," in *ISMIR*, 2014, vol. 10, p. 2014.

[27] Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Mohammad Norouzi, Douglas Eck, and Karen Simonyan, "Neural audio synthesis of musical notes with wavenet autoencoders," in *International Conference on Machine Learning*. PMLR, 2017, pp. 1068–1077.

[28] George Tzanetakis and Perry Cook, "Musical genre classification of audio signals," *IEEE Transactions on speech and audio processing*, vol. 10, no. 5, pp. 293–302, 2002.

[29] Geoffroy Peeters and Joachim Flocon-Cholet, "Perceptual tempo estimation using gmm-regression," in *Proceedings of the second international ACM workshop on Music information retrieval with user-centered and multimodal strategies*, 2012, pp. 45–50.

[30] Stephen W Hainsworth and Malcolm D Macleod, "Particle filtering applied to musical tempo tracking," *EURASIP Journal on Advances in Signal Processing*, vol. 2004, pp. 1–11, 2004.

[31] Matthew C McCallum, Florian Henkel, Jaehun Kim, Samuel E Sandberg, and Matthew EP Davies, "Similar but faster: manipulation of tempo in music audio embeddings for tempo prediction and search," *arXiv preprint arXiv:2401.08902*, 2024.

[32] Elio Quinton, "Equivariant Self-Supervision for Musical Tempo Estimation," in *International Society for Music Information Retrieval Conference (ISMIR)*. Sept. 2022, number arXiv:2209.01478, arXiv.

[33] Jeong Choi, Seongwon Jang, Hyunsouk Cho, et al., "Towards proper contrastive self-supervised learning strategies for music audio representation," in *2022 IEEE International Conference on Multimedia and Expo (ICME)*. IEEE, 2022, pp. 1–6.

Appendix for the paper "Leave-One-EquiVariant: Alleviating invariance-related information loss in contrastive music representations" — preprint only

V. Appendix

A. Training protocol comparison with original MULE

We report differences in training protocols, training scales, and model architectures in Tab. III to explain possible discrepancies between our results and the results reported in [5].

		MULE-Large	MULE-Small	Ours
Architecture	#Params	63M	63M	53M
	Proj. head	[4096,4096,4096,1728]		[1728,1728,512]
Data	Hours	117.5k	5k	2.9k
	Samples	-	1.8M	55k
Hyperparams	batch size	3840	512	256
	GPUs	16 A100 80GB	16 A100 80GB	2 A5000 24GB
	Precision	-	-	Mixed 16bit

TABLE III: Training and architecture differences between official MULE implementation [5] and ours.

Despite these differences, we manage to reach comparable performance to MULE on a range of downstream tasks mentioned in the paper as shown in Tab. I when training with PSTS, PS or TS. Because we are not attempting to surpass the official implementation of MULE but rather to demonstrate the capability of our method when compared to similarly scaled and trained models, this similar performance serves more as a sanity check and is satisfactory within the scope of this study.

B. Specifics of augmentation chains for pretraining

Tab. IV reports the base augmentation chain $\mathcal{T}_B$ and variant augmentation chain $\mathcal{T}_V$ as well as maximum and minimum transformation parameters:

Augmentation	probability	parameter	Min/Max	unit
$\mathcal{T}_B$				
Gain	0.7	Gain	-15 / 5	dB
Polarity inv.	0.8	-	-	-
Colored Noise	0.8	Signal/noise ratio	3 / 30	dB
		Spectral decay	-2 / 2	dB/octave
Filtering	(One of)			
Low pass	0.5	Cutoff	0.15 / 7	kHz
High pass	0.5	Cutoff	0.2 / 2.4	kHz
Band pass	0.5	center frequency	0.2 / 4	kHz
		Bandwidth fraction	0.5 / 2	-
Band cut	0.3	center frequency	0.2 / 4	kHz
		Bandwidth fraction	0.5 / 2	-
Reverb	0.5	room size	0.2 / 1	-
		wet factor	0 / 1	ratio
Distortion	0.6	Drive	1 / 10	dB
$\mathcal{T}_V$				
Pitch shifting	0.5	transpose	-4 / 4	semitones
Time stretching	0.5	transpose	0.7 / 1.3	ratio

TABLE IV: Training augmentation chains - Variant ($\mathcal{T}_V$) and Base ($\mathcal{T}_B$). We generate 4 augmentation for each anchor.

C. MULE Architecture, LOEV++ Architecture

Tab. V Shows the architecture described in previous work [5], [19]. We show the modified LOEV++ architecture in Fig. 4 - Half of the last block 4 of the SF-NF0Net architecture is parallelized into 3 blocks and prepended to the projection heads g^i, g^p, g^t. This changes our models' parameter count from 53M to 78M. Note that in LOOC [16], parallelizing the *conv5* block of the ResNet architecture close to *triples the size of the network*. This might explain why the disentanglement results we obtain in Table II and Fig. 2 can be perceived as modest. However, we wish to remain computationally sober when compared to MULE while achieving desirable disentanglement results. We could increase the portion of the network that is resorbed into the projection heads, but would lose that computational sobriety, and one might argue that training different contrastive models — one per variant augmentation — might be more sensible if we were to forgo all compute limitations.

Stage	Slow Path	Fast Path	$T \times F$
Spectrogram	-	-	400×128
data layer	stride 4,1	stride 1,1	Slow : 100×128 Fast : 400×128
Stem 1	1×3, 16 stride 1,1	3×3, 2 stride 2,2	Slow : 50×64 Fast : 200×64
Stem 2	1×3, 32 stride 1,1	3×3, 4 stride 1,1	Slow : 50×64 Fast : 200×64
Stem 3	1×3, 64 stride 1,1	3×3, 8 stride 1,1	Slow : 50×64 Fast : 200×64
Stem 4	3×3, 128 stride 2,2	3×3, 16 stride 2,2	Slow : 25×32 Fast : 100×32
Block 1	$\begin{bmatrix} 1 \times 1, 128 \\ 1 \times 1, 128 \\ 1 \times 3, 128 \\ 1 \times 1, 256 \end{bmatrix} \times 1$	$\begin{bmatrix} 1 \times 1, 16 \\ 1 \times 1, 16 \\ 1 \times 3, 16 \\ 1 \times 1, 32 \end{bmatrix} \times 1$	Slow : 25×32 Fast : 100×32
Block 2	$\begin{bmatrix} 1 \times 1, 256 \\ 1 \times 1, 256 \\ 1 \times 3, 256 \\ 1 \times 1, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 1 \times 1, 32 \\ 1 \times 1, 32 \\ 1 \times 3, 32 \\ 1 \times 1, 64 \end{bmatrix} \times 2$	Slow : 25×16 Fast : 100×16
Block 3	$\begin{bmatrix} 1 \times 1, 768 \\ 1 \times 1, 768 \\ 1 \times 3, 768 \\ 1 \times 1, 1536 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 96 \\ 1 \times 1, 96 \\ 1 \times 3, 96 \\ 1 \times 1, 192 \end{bmatrix} \times 6$	Slow : 25×8 Fast : 100×8
Block 4	$\begin{bmatrix} 1 \times 1, 768 \\ 1 \times 1, 768 \\ 1 \times 3, 768 \\ 1 \times 1, 1536 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 96 \\ 1 \times 1, 96 \\ 1 \times 3, 96 \\ 1 \times 1, 192 \end{bmatrix} \times 3$	Slow : 25×4 Fast : 100×4
Global average pool & concatenate			$d_E = 1728$

TABLE V: F0-SF-NFNet architecture as described in [19].

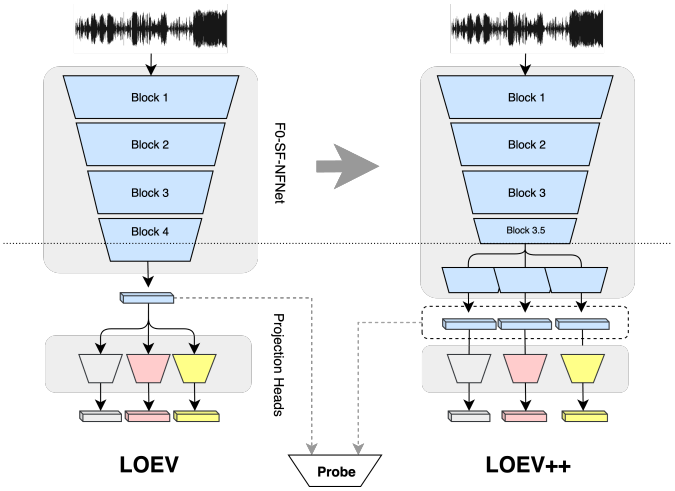

Fig. 4: LOEV(++) architectures. In either case, the probing representation can come from the embedding superspace, or, for LOEV++, the concatenation of the parallel superspaces can be used, as in [16]

D. Training protocol discussion

While our method is reasonably simple, it depends on a range of design choices which might impact its capability in learning robust representations and conserving information from all augmentations. The impact of some of these design choices will be explored, but some are too computationally expensive to make a comprehensive and exhaustive exploration of them feasible. A few straightforward examples are as follows:

Variant augmentation application probability : Stronger augmentations are *generally* beneficial to coarse music information retrieval tasks such as automatic tagging, as shown in previous work. In our case, for variant augmentations, as the probability of application gets higher, the contrastive matrix for a given head gets sparser, with certain application leading to an empty contrastive matrix, which is not desirable. There is certainly a balance to strike in terms of probability of application of each augmentation.

Similarity strategy: Briefly mentioned in Section III-B, the manner in which we consider augmented samples to be similar or not is an important design choice. If we simply take non-augmented pairs as positive, this does not cover same-parameter-augmented pairs as positives, which they are. Further, perceptually-similar augmented samples are treated the same as radically different augmented samples with our strategy, which might introduce confusion into the training objective.

Number of augmentations in $\mathcal{T}_V$: As the number of variant augmentations gets higher, the model stores more relevant information about some of these augmentations, which might be beneficial for some downstream tasks. However, it seems unlikely that all possible augmentations can be accounted for, computationally, and from a model size standpoint. The more augmentations are included as variant augmentations, the more information the model must store about these augmentations, information which might be contradictory, and the necessary disentanglement limited by model size. Because we limit our study to two semantically-evident augmentations in the field of MIR, we do not consider this design choice to fall within the scope of this study.

In-track sampling strategy. The position at which the positive pairs are taken within the track is also important in our case. MULE adopts an all-track sampling strategy where positive chunks can be sampled from anywhere within the track. In our case, we wish to preserve key and tempo information between two positives to avoid confusing the model with false positives. Indeed, key and tempo are often used to globally describe a track, but they are local properties that might change throughout a track. Tempo changes are relatively rarer, but key changes are fairly common. However, all-track sampling has been shown to be more robust for general music understanding [33]. To strike a balance, we design a sampling strategy where positives can either be sampled from *the same* chunk, *adjacent* chunks, or *anywhere within the track*. By defauly these strategies are sampled uniformly but this is a debatable design choice, which might prove to deteriorate our variant head objectives.